

RESEARCH ARTICLE

Open-Domain Factoid Question-Answering in Urdu: Data and Methods

MUHAMMAD SHAKEEL^{ID} AND RAO MUHAMMAD ADEEL NAWAB

Department of Computer Science, COMSATS University Islamabad, Lahore Campus, Lahore 54000, Pakistan

Corresponding author: Muhammad Shakeel (fa18-pcs-001@cuilahore.edu.pk)

This work was done with support from Dr. Ashraf Habibullah, president and CEO of Computers and Structures, Inc. (USA).

ABSTRACT Open-domain factoid question-answering (ODFQA) aims to answer questions posed in natural language by retrieving and extracting relevant information from large, unstructured text sources. A range of applications have benefited from ODFQA including improved search relevancy in search engines and information retrieval, enable semantic searches, interactive and personalized learning systems, and the creation of large language models. Although significant research on ODFQA for English and other languages has been done, Urdu-specific research in this area remains limited. This is due to the lack of high-quality datasets and the challenges associated with processing Urdu text. To address the unavailability of Urdu-specific resources for the ODFQA task, we developed a benchmark corpus, comprising 3,985 Urdu questions and corresponding Urdu Wikipedia articles, with 1,006 answerable and 2,979 unanswerable questions. Each question in our proposed corpus was manually annotated by three independent annotators. As a secondary contribution, we carried out extensive experimentation using a range of state-of-the-art models, including retrievers (BM25 and Sentence-BERT), multilingual transformers (mBERT, XLM-RoBERTa-Large, XLM-RoBERTa-Large-Squad2), and large language models (GPT-3.5-Turbo-0125, GPT-4o-mini-2024-07-18) on our proposed corpus. Best results were obtained using the XLM-RoBERTa-Large-Squad2 model with $F_1 = 0.61$ and $EM = 0.41@k = 20$. While the finetuned GPT-4o-mini model was the best model, with $F_1 = 0.81$ and $EM = 0.81@k = 1$. To foster research in the Urdu ODFQA, our proposed corpus is freely available under the Creative Commons license.

INDEX TERMS Corpus, factoid, GPT, large language models, multilingual, open-domain, question-answering, RAG, transformers, Urdu.

I. INTRODUCTION

Open-domain factoid question answering (ODFQA) is a natural language processing (NLP) task that requires understanding natural language questions, retrieving relevant information from large text sources, and extracting accurate short answers or facts [1]. ODFQA, with its potential applications in information retrieval, customer service, virtual assistants, education, and research, is crucial to facilitate informed decision making. Despite significant advancements, and availability of free large text sources, ODFQA remains a challenging task due to the inherent difficulties in understanding natural language, retrieving relevant information from

large-scale datasets, and identifying the correct answer span within the retrieved text.

The problem of ODFQA has been mainly explored in English and some other languages [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]. However, the application of ODFQA to South Asian languages, particularly Urdu, has not been well studied. In literature, we have found only two Urdu corpora, UQuAD1.0 [12] and UQA [13], for the ODFQA task. The main limitations of these two corpora are: (1) These two corpora are based on the translation of English SQuAD-v1 [4] and SQuAD-v2 [5] datasets, limiting their ability to capture the complexities of real-world Urdu queries, (2) Both corpora focus on shorter Wikipedia passages, potentially simplifying the task and limiting its applicability to more complex, real-world scenarios, and (3) The annotation process for these corpora may not have followed rigorous standards

The associate editor coordinating the review of this manuscript and approving it for publication was Arianna DULIZIA^{ID}.

of applying multiple annotations per question, potentially impacting data quality and reliability. To overcome the limitations of existing Urdu corpora for the ODFQA task, this study aims to develop a benchmark corpus and state-of-the-art methods for Urdu ODFQA task.

Urdu is a popular South Asian language with a rich cultural heritage, plays a key role in communication, education, and cultural exchange.¹ Its comprehensive grammar, extensive vocabulary, and rich poetic heritage make it a fascinating language to study and appreciate. Despite its widespread use, Urdu is still considered a low-resource language for NLP research due to the limited availability of large-scale, annotated datasets and pre-trained models [14]. Developing Urdu question-answering systems is essential for preserving its cultural heritage, enhancing information access, supporting education, and advancing technological development of the language. By contributing to the Urdu ODFQA task, we can contribute to the development and advancement of the Urdu language in the field of NLP.

There is a lack of benchmark Urdu ODFQA corpora that represents real-world information needs and ensures that the questions are representative of actual user intent. To bridge this gap, there are two primary objectives of this study: (1) to develop a corpus that addresses this need for the Urdu ODFQA, and (2) to develop, evaluate, and compare state-of-the-art retriever, transformer, and large language models on the proposed corpus. To address the first objective, we propose UrFQuAD-v1, a benchmark corpus for the Urdu ODFQA task. For the second objective, we aim to apply and compare state-of-the-art models on the UrFQuAD-v1 corpus.

The proposed UrFQuAD-v1 corpus could become an important element in the following applications: (1) Evaluate and compare the performance of existing methods for the Urdu ODFQA, (2) Develop new techniques specifically tailored for the Urdu ODFQA, (3) Analyze the challenges and opportunities presented by the Urdu language for the ODFQA task, (4) Building search engines and information retrieval systems for the Urdu content, and (5) promote and support further research in the Urdu ODFQA task.

The rest of this paper is organized as follows: Section II highlights the available corpora and techniques for the ODFQA task. Section III explains the creation of the proposed benchmark UrFQuAD-v1 corpus. Section IV describes the state-of-the-art retriever, transformer, and large language models applied on the proposed UrFQuAD-v1 corpus. Section V outlines the experimental setup. Section VI discusses the results and their analysis. Finally, Section VII concludes the paper by summarizing key findings and potential future directions.

II. RELATED WORK

Open-domain factoid question-answering (ODFQA) has witnessed significant research interest, driven by the availability of extensive benchmark corpora, and the advancements

in neural network models. This section explores in detail the characteristics of various ODFQA benchmark datasets, including their development processes and techniques.

A. CORPORA FOR ODFQA

While ODFQA for the English-language has been extensively studied, there is a growing interest in exploring this field for other languages as well. In [3], the authors proposed the WikiQA corpus for the ODFQA task. This dataset contains 3,047 questions and was created using real-world Bing search queries. Each question was paired with a relevant Wikipedia passage. Questions were identified using heuristics based on question words and question marks. The dataset included both answerable and unanswerable questions. The CNN with Count Features [15] model achieved the best performance on WikiQA, with (EM = 0.39, and F1 = 0.49) scores.

In [4], the authors proposed the SQuAD (Stanford Question Answering Dataset) for the ODFQA task. It is a benchmark dataset for machine reading comprehension, consisting of 107,785 question-answer pairs derived from Wikipedia articles. Crowd workers posed questions, and answers are spans of text within the corresponding articles. The questions cover a wide range of topics from Wikipedia. Various techniques were evaluated on SQuAD-v1.1, with Logistic Regression resulting as the best model, with a (F1 = 0.51) score. In [5], the authors introduced SQuAD-v2, an extension of SQuAD-v1.1, to create a more challenging dataset for the ODFQA task. By adding 53,775 unanswerable questions designed to resemble answerable ones, SQuAD-v2 aimed to evaluate models' ability to identify when questions cannot be answered based on the provided context. EM and F1 scores were used for evaluation. A combination of DocQA [16] and ELMo [17] model achieved the best performance, with (EM = 0.63, and F1 = 0.66) scores on the test set.

In [6], the authors propose SearchQA corpus for the ODFQA task. This dataset consists of 140,461 QA pairs and is derived from Jeopardy! (American TV quiz show) questions. Questions were submitted to Google Search, and answers were extracted from the top 40 retrieved documents. The dataset features short, exact answers of 1-2 words across diverse topics. Evaluated using accuracy and F1-score, the Attention Sum Reader (ASR) [18] model achieved the best performance with (accuracy = 0.41, and F1 = 0.228) scores on the test set.

In [7], the authors proposed the TriviaQA corpus for the ODFQA task. This dataset contains 95,956 QA pairs and was created by trivia enthusiasts with supporting evidence from the Web and Wikipedia. Trivia-Web used the top 10 Bing search results as context for each question, while Trivia-Wiki used all relevant Wikipedia pages. Evaluated using EM and F1-score, the BiDAF (Bidirectional Attention Flow) [19] model achieved the best performance with (EM = 0.49, F1 = 0.558) scores on the test set.

In [2], the authors proposed the Natural Questions corpus for the ODFQA task. It is a dataset of 323,045 anonymized

¹<https://www.ethnologue.com/language/urdu/> Last visited: 21-July-2022

Google search queries. Human annotators identified various forms of answers within relevant Wikipedia passages for each query. These answers included short answers (5 tokens or fewer), long answers (typically paragraphs), and no answer. Covering a wide range of topics, Natural Questions is a challenging dataset requiring models to understand and answer questions based on entire Wikipedia articles. Publicly available, researchers have used it to train and evaluate question-answering systems. Evaluated using the Precision, Recall, and F1-score, the DocumentQA [16] model achieved the best performance with (Precision = 0.406, Recall = 0.31, and F1 = 0.35) scores in the short answer category on the Natural Questions' test set.

Researchers also applied a cross-lingual approach to develop ODFQA systems, addressing the challenge of limited datasets for low-resource languages. In [8], the authors proposed the XQA corpus for the ODFQA task. The dataset is a cross-lingual dataset of 90,610 QA pairs extracted from Wikipedia's "Did you know" sections across nine languages: English, French, German, Portuguese, Polish, Chinese, Russian, Ukrainian, and Tamil. Evaluated using EM and F1-score, BERT [20] model achieved the best performance for English, with (EM = 0.337, and F1 = 0.405) scores on the test set.

In [21], the authors propose MLQA, a benchmark for cross-lingual corpus for the ODFQA task. The dataset covers seven languages: English, Arabic, German, Spanish, Hindi, Vietnamese, and Simplified Chinese. Containing over 12,000 instances in English and 5,000 in each other language, MLQA evaluates state-of-the-art cross-lingual models and machine-translation-based baselines. These include the BERT-Large [20], mBERT [20], and XLM-RoBERTa [22] models. For the English language, BERT-Large achieved the best performance with (F1 = 0.80 and EM = 0.67). mBERT was the best performing model for the Hindi (F1 = 0.55, EM = 0.40), Vietnamese (F1 = 0.62, EM = 0.43), and Simplified Chinese (F1 = 0.61, EM = 0.39) languages. While the XLM-RoBERTa model performed best for the Spanish (F1 = 0.68, EM = 0.49), Arabic (F1 = 0.54, EM = 0.36), and German (F1 = 0.62, EM = 0.47) languages. In [23], the authors proposed the MKQA corpus for the ODFQA task. The dataset is a comprehensive open-domain dataset comprising 260,000 question-answer pairs across 26 typologically diverse languages (10,000 pairs each). English Wikipedia and the specific language's Wikipedia served as external knowledge sources. Evaluated using F1-score, the XLM-RoBERTa (Cross-lingual RoBERTa) [22] achieved the best performance with an (F1 = 0.429) score on the translated test set.

Despite significant advancements in English question-answering, research on monolingual non-English corpora, particularly for low-resource languages, has been limited. This disparity is primarily due to the scarcity of annotated data and computational resources for these languages. For example, in [9], the authors propose PQuAD,

a Persian language corpus, for the ODFQA task. The dataset is a crowdsourced reading comprehension dataset based on Persian Wikipedia articles. Comprising 80,000 question-answer pairs, PQuAD includes 25% unanswerable questions to enhance challenge. The authors applied various state-of-the-art pre-trained contextualized language models, and the best performance was achieved by the XLM-RoBERTa model with (EM = 0.748, and F1 = 0.876) scores.

In [10], the authors propose the UIT-ViQuAD, a Vietnamese language corpus for the ODFQA task. The dataset comprises over 23,000 human-generated question-answer pairs based on Vietnamese Wikipedia articles, which are based on 5,109 passages from 174 Vietnamese Wikipedia articles. The authors applied various deep learning methods, and the best performance was achieved by the XLM-RoBERTa-Large model with (EM = 0.68, and F1 = 0.87) scores on the test set.

In [11], the authors proposed ArQuAD, an Arabic language corpus, for the ODFQA task. The dataset comprises 16,020 questions posed by language experts on Arabic Wikipedia passages. Each question has a corresponding answer segment within the article. The authors conducted various analyses to understand ArQuAD's characteristics and established benchmarks by finetuning pre-trained language models. The model AraBERTv0.2-large [24] achieved the best performance with (EM = 0.68 and F1 = 0.87) scores.

For the Urdu ODFQA task, there are considerable challenges in the Urdu Natural Language Processing. These originate from the language's inherent characteristics, including its complex and morphologically rich nature, right-to-left script, and the ambiguity introduced by often omitted vowel markings. Adding to these linguistic complexities is the lack of annotated datasets and standardized resources as compared to languages like English, that greatly impede the development of robust Urdu NLP models. Furthermore, the diverse landscape of Urdu dialects and frequent word mixing with regional languages adds more complexity to model training and evaluation, often requiring specialized tokenization and normalization techniques. Despite these challenges, advancements in Urdu NLP promise transformative real-world applications. These applications could include enhanced educational platforms tailored to Urdu speakers, healthcare systems empowered by efficient patient data processing, improved information retrieval within government and community services, sophisticated Urdu sentiment analysis, seamless online communication for Urdu-speaking communities, and Urdu chatbots.

Due to these challenges, question-answering resources remain limited for the Urdu ODFQA task. In [12] the authors proposed UQuAD1.0, an Urdu language corpus for the Urdu ODFQA task. The corpus has been developed using a translated version of the SQuAD-1.1 [4], comprising 49,000 question-answer pairs. Out of these, 45,000 were translated from SQuAD-1.1, while the remaining 4,000 were generated

using crowdsourcing. The XLM-RoBERTa model achieved the best performance with ($F1 = 0.66$) score.

In [13], the authors proposed UQA, an Urdu language corpus, for the ODFQA task. The dataset was created by translating SQuAD-2.0 [5] using the EATS (Enclose to Anchor, Translate, Seek) technique. This ensured answer spans were preserved in the translated context paragraphs. The authors applied state-of-the-art multilingual QA models on UQA, including mBERT [20], XLM-RoBERTa, and mT5 [25]. The XLM-RoBERTa-XL model achieved the best performance with ($EM = 0.74$ and $F1 = 0.85$) scores.

B. TRANSFORMER AND LARGE LANGUAGE MODELS FOR THE ODFQA

Researchers have developed various techniques and methods using the aforementioned corpora to address the open-domain factoid question-answering (ODFQA) task. In [26], the authors used a dual-encoder framework, based on BERT (base, uncased) [20], for their dense passage retrieval approach. This framework consists of two separate neural networks: one for encoding questions and another for encoding passages. The encoded representations are then compared using cosine similarity to retrieve relevant passages for a given question. The authors applied their technique to the Natural Questions [2], TriviaQA [7], and SQuAD-v1.1 [4] datasets. The best results were obtained on the Natural Questions dataset, with scores of (Accuracy = 79.4 @ $k = 20$ and Accuracy = 86.0 @ $k = 100$).

In [27], the authors demonstrated that large language models (LLMs) can effectively learn new tasks with limited data. They applied GPT-3, a 175 billion parameter autoregressive language model, to various NLP tasks, including open-domain question-answering (ODQA), translation, common sense reasoning, and reading comprehension, in zero-shot, one-shot, and few-shot settings. For ODFQA, GPT-3 was evaluated on Natural Questions and TriviaQA datasets. On TriviaQA, GPT-3 achieved an accuracy of 0.68 in one-shot and 0.71 in few-shot settings, demonstrating its ability to perform well even with limited task-specific examples.

In [28], the authors introduce new high-quality datasets for Answer Sentence Selection (AS2) in five European languages: French, German, Italian, Portuguese, and Spanish. AS2 involves identifying the most relevant sentence or passage within a document that contains the answer to a given question. These datasets were created using supervised Automatic Machine Translation (AMT) of existing English AS2 WikiQA [3] dataset, leveraging a Large Language Model (LLM). Various Transformer architectures, such as mBERT (multilingual BERT) [20], and XLM-RoBERTa [22] were used to evaluate the quality of the translated datasets and the performance of AS2 models. Results are reported in terms of Mean Average Precision (MAP) and Precision@1 ($P@1$) metrics. The best results for the English ($MAP = 0.85$, $P@1 = 0.79$) and German ($MAP = 0.84$, $P@1 = 0.77$)

languages were achieved by the XLM-RoBERTa model. While the best results for the French ($MAP = 0.86$, $P@1 = 0.80$), Italian ($MAP = 0.84$, $P@1 = 0.76$), Portuguese ($MAP = 0.84$, $P@1 = 0.76$), and Spanish ($MAP = 0.85$, $P@1 = 0.77$) languages were achieved by the mBERT model.

In [29], the authors introduce a framework that leverages multilingual knowledge. By assembling triples from different languages to form a multilingual knowledge base and incorporating this knowledge into pre-trained language models, the authors significantly improved performance on the MLQA [21] dataset. This demonstrates the effectiveness of their approach in addressing the challenge of limited knowledge transfer between languages in multilingual question answering. The authors applied the XLM-RoBERTa model, and the best results were achieved on the German language with ($F1 = 0.63$, $EM = 0.50$) scores.

To summarize, the researchers have made efforts to develop standard corpora and methods for the ODFQA task. Majority of these efforts focus on English and other languages. We have found two studies [12], [13], that have developed corpora for the Urdu ODFQA systems. However, these Urdu corpora have a number of limitations: (1) These corpora are based on the translation of the English SQuAD-v1 [4] and SQuAD-v2 [5] corpora, thus, may not accurately capture real-world Urdu queries, (2) The question contexts are short Wikipedia passages that may not reflect the complexities of real-world information retrieval from large documents, and (3) Due to the lack of multiple annotations per question, the annotations may be influenced by annotator biases, potentially affecting data reliability. To address these limitations, we propose a benchmark manually annotated corpus for the Urdu ODFQA tasks (see Section III for details of the proposed corpus). As far as we are aware, no previous studies have reported the development of a standard benchmark Urdu ODFQA corpus.

III. CORPUS GENERATION PROCESS

This section presents the development process of our proposed open-domain factoid question-answering dataset in Urdu, hereafter named as UrFQuAD-v1 (Urdu Factoid Question-Answering Dataset, version-1), including the description of baseline corpus, task description, data selection and collection from baseline corpus, data cleaning and pre-processing, annotation process, and corpus characteristics and standardization. In the next sections, we will explain the corpus development process in detail.

A. BASELINE CORPUS

This work prioritizes developing an open-domain factoid question-answering dataset for Urdu. To develop our proposed UrFQuAD-v1 corpus for the Urdu language, we chose the benchmark *Natural Questions* corpus in English language [2], henceforth named as NQ, as the baseline corpus. The NQ corpus is a diverse collection of real-world English questions with corresponding answers and serves as a benchmark dataset for developing open-domain factoid, as well

as question-answering systems in English that emphasize understanding and retrieving information from real-world, complex queries that users might ask. To create the NQ corpus, the authors leverage anonymized and aggregated search queries submitted to the Google search engine.² These real-world queries serve as the foundation for the question set within the NQ corpus. Given each question, human annotators receive corresponding Wikipedia pages from the top 5 Google search results and are tasked with manually annotating both *long answers* (questions whose answers span a passage within the page), and *short answers* (questions whose answers consist of one or more entities found on the page), or mark *NULL* as an answer if an answer cannot be found. The authors also implement a rigorous evaluation process to ensure the quality of the annotations for each question. The NQ corpus is then standardized as compressed JSON Lines (jsonl) files, and distributed³ as *train* (307, 373 questions), *test* (7, 842 questions), and *development* (7, 830 questions) sets.

The NQ corpus serves as an established benchmark for creating new QA datasets [23], [30], [31]. These works leverage the strengths of NQ, including its high-quality human annotations, standardized structure, and open-access availability, to create diverse QA datasets. For our task, we focused on the 106, 926 factoid questions within the NQ training set, which represent a sizable portion (approximately 34.79%) of the total 307, 373 training questions. These factoid questions align perfectly with our goal of developing UrFQuAD-v1, a factoid QA dataset in Urdu. Also, NQ's questions originate from real-world search queries, reflecting actual user information needs. Moreover, the broad topic coverage of NQ's questions demonstrating practical relevance, also aligns with the open-domain nature of UrFQuAD-v1 directly supporting our goal. Finally, the NQ corpus contains high-quality human annotations that ensure accuracy and its standardized structure, along with open-access availability, facilitates efficient adaptation and translation into Urdu. By leveraging this subset of NQ, we aim to address the lack of high-quality Urdu QA datasets and contribute to the developments in the multilingual NLP research.

B. DATA SELECTION FROM THE NQ CORPUS

The NQ corpus comprises 307, 373 questions in the train set, and 7, 842 questions in the test set. From the 307, 373 questions, we selected 106, 926 questions for this study. The reason for selecting this subset of questions is that these questions are annotated as having *short answer* in the NQ corpus, thus, specifically target short or factoid questions. This targeted selection aligns with the open-domain factoid question-answering nature of our proposed UrFQuAD-v1 corpus.

After the selection of 106, 926 questions in English, we translated these into Urdu in the following steps: (1) we extracted the required set of 106, 926 questions from the NQ's train set,⁴ and (2) we then used the *Google Batch Translation*⁵ service to automatically translate the selected set of 106, 926 factoid English questions into Urdu.

To address the challenges, in addition to the inherent time and resource constraints associated with manual translation of the large number of 106, 926 questions, we employed the automated Google Batch Translation service as an efficient initial step. Prior to the full translation, we conducted a preliminary study in 2022 to assess the effectiveness of different translation services. This involved translating a sample of 1, 000 factoid questions from the NQ corpus using both Google Batch Translation and Bing Translator.⁶ As illustrated by a small result sample in Figure 1, our evaluation indicated that Google Batch Translation delivered better results for translating short Urdu texts. Based on these findings, we opted to leverage Google Batch Translation as the initial step in the UrFQuAD-v1 creation process, ensuring an efficient and effective approach.

C. DATA SOURCES SELECTION FOR SEARCHING ANSWERS

To search for the answers of the question selection from the UrFQuAD-v1 corpus, we chose the entire Urdu Wikipedia.⁷ Our selection is based on the following reasons: (1) Urdu Wikipedia offers a large collection of articles in various domains (open-domain) providing a rich source of potential documents to search answers to questions, and (2) Wikipedia articles are freely available, ensuring alignment with our research goals.

For our work, we used the Urdu Wikipedia dump file⁸ dated July 20, 2022. This dump file comprises 369, 143 Urdu articles. These articles were cleaned by removing any HTML markups and excess white spaces.

To identify the answer to each translated factoid question from the Urdu Wikipedia collection, we used the *Google Custom Search JSON API*.⁹ We used this API because it facilitated search in our task as explained shortly. To use this API, it is first configured in two steps: (1) setup a Programmable Google Search Engine that is configured to prioritize search in the user-defined domains, with an optional limit on top *n* responses against each request, and (2) provide search requests to this search engine, and retrieve JSON formatted objects against each request that contains the search request, and the defined top *n* search responses. It is to be noted that the Programmable Search Engine returns

⁴https://storage.cloud.google.com/natural_questions/v1.0-simplified/simplified-nq-train.jsonl.gz Last visited: 21-July-2022

⁵<https://cloud.google.com/translate/docs/advanced/batch-translation> Last visited: 21-July-2022

⁶<https://www.microsoft.com/en-us/translator/> Last visited: 21-July-2022

⁷<https://ur.wikipedia.org/> Last visited: 21-July-2022

⁸<https://dumps.wikimedia.org/urwiki/latest/> Last visited: 21-July-2022

⁹<https://developers.google.com/custom-search/v1/overview> Last visited: 21-July-2022

²<https://www.google.com/> Last visited: 21-July-2022

³<https://ai.google.com/research/NaturalQuestions/download> Last visited: 21-July-2022

Sr. No.	NQ English Factoid Question	Google Urdu Translated Text
1	who had the most wins in the nfl	جس نے سب سے زیادہ جیت NFL میں حاصل کی تھی
2	who is the current president of un general assembly	جو اقوام متحدہ کے موجودہ صدر ہیں
3	when do the eclipse supposed to take place	چاند گرہن کب ہونا چاہئے؟
4	what is the name of the sea surrounding dubai	دبئی کے آس پاس کے سمندر کا کیا نام ہے؟
5	who holds the nba record for most points in a career	جو کیریئر میں سب سے زیادہ پوائنٹس کے لئے این بی اے ریکارڈ رکھتا ہے
6	how many players on a box lacrosse team	باکس لیکرس ٹیم میں کتنے کھلاڑی ہیں
7	where are the upcoming olympics to be held	آنندہ اولمپکس کہاں منعقد ہوں گی
8	who plays norman bates in the tv show	جو ٹی وی شو میں نورمن بیٹس کھیلتا ہے
9	when did usa start driving on the right	جب USA نے دائیں طرف سے گاڑی چلانی شروع کی
10	name some components of the central nervous system (cns)	مرکزی اعصابی نظام کے کچھ اجزاء کا نام بتائیں (cns)

FIGURE 1. Google batch translation - Sample results of 10 questions.

responses prioritizing the user-defined sources and includes results from the other matching relevant sources.

For our task, this API was configured as follows: (1) the Programmable Google Search Engine was configured to search using the Urdu Wikipedia domain as the primary source, with a defined limit of top 10 results against each request, and (2) the translated 106,926 question set (see Section III-B) was provided as the search requests, and JSON objects consisting of the question (search request), and the top 10 URLs of the documents (search response) that are the potential answer sources of the respective question were retrieved.

We then applied filtering on the returned search results, so that the results having source URLs belonging to the Urdu Wikipedia were selected, and the other results were discarded. Through this filtering process, we identified a pool of 39,485 questions with corresponding answer sources within the Urdu Wikipedia, forming the basis of the UrFQuAD-v1 factoid question set.

The probable reason for not finding Urdu Wikipedia articles (containing an answer to a question), for a large pool of questions, is that the size (number of documents) of the Urdu Wikipedia is quite small as compared to the English Wikipedia.

D. DATA PRE-PROCESSING

After identifying the potential Urdu Wikipedia articles against each of the 39,485 Urdu factoid questions, we then linked the questions and articles. Our analysis showed that in many situations, a single article contained answers to multiple questions. Consequently, the number of Urdu Wikipedia articles containing answers to the questions was significantly reduced to 4,246.

We also annotated each Urdu Wikipedia article at four levels of granularity: (1) document-level, (2) passage-level, (3) sentence-level, and (4) answer text. This annotation was applied as follows: (1) each article's name was replaced with a unique identifier, (2) assigning passage numbers within each article, (3) utilizing *Stanford Stanza*¹⁰ library for optimal Urdu sentence segmentation. Each sentence then received

a unique identifier based on its location within the article and the corresponding passage, and (4) the actual answer text annotation was addressed during the manual annotation phase using our custom annotation Java application (see Section III-E3).

E. ANNOTATION PROCESS

To ensure the creation of a high-quality and reliable Urdu factoid question-answering dataset (UrFQuAD-v1), we implemented a meticulous and structured annotation process. This process involved several key steps: (1) Annotation Guidelines, (2) Selection and Training of Annotators, (3) Annotation Execution, (4) Ensuring Annotator Consensus, and (5) Annotation Process Outcome. Each step was carefully designed to enhance the reliability and validity of the dataset.

1) ANNOTATION GUIDELINES

Developing comprehensive annotation guidelines was a critical first step to standardize the annotation process across all annotators. The guidelines provided detailed instructions on: (1) Identification of Answerable and Unanswerable Questions, (2) Selection of Categories and Entity Types, (3) Instructions on assigning the most appropriate category and entity type for each question, and (4) Annotation Procedures. These guidelines aimed to ensure consistency in annotations and overcome potential errors arising from individual interpretations. To simplify the annotation classification, we developed the following annotation guidelines:

1) Annotating Answerable Questions

- Identify the most appropriate *Category* (domain) that best reflects the subject of the question.
- Identify the most relevant *Entity* type that best corresponds to the target entity the question focuses on.
- Carefully examine the entire Urdu Wikipedia article's data provided, paying close attention to all relevant information. Based on this thorough review, identify the answer to the question that best satisfies the two key criteria: *Category* and the *Entity* types. The answers should be directly related to both the question's category and the chosen entity type, and should

¹⁰<https://stanfordnlp.github.io/stanza/> Last visited: 21-July-2022

be comprehensive and concise, providing enough information to completely answer the question.

- d) If a suitable answer is found, then select the corresponding text from the article's text that directly represents this answer and place it in the designated *Answer* box.

2) Annotating Unanswerable Questions

- a) Repeat steps *a*, *b*, and *c* as given above.
- b) If the answer cannot be found within the provided context, mark *No Answer* and record a *NULL* value in the designated *Answer* box.

2) SELECTION AND TRAINING OF ANNOTATORS

A team of 40 native Urdu-speaking annotators was assembled to carry out the annotation task. The annotators were primarily Master's and Ph.D. students from the Computer Science department at COMSATS University,¹¹ selected for their strong comprehension skills and expertise in the Urdu language.

To ensure high-quality annotations, we conducted comprehensive training sessions for all annotators. These sessions covered the annotation guidelines and the use of our custom Java annotation application (see Section III-E3). Hands-on practice with sample data allowed annotators to familiarize themselves with the annotation process and clarify any uncertainties. We strongly encouraged annotators to thoroughly understand the guidelines to ensure consistency and accuracy in their annotations.

3) ANNOTATION EXECUTION

To ensure high-quality annotations, we employed a rigorous three-cycle annotation process over a year (August 2022 to August 2023) utilizing a **triple annotation** approach (see Section III-E4). The pool of 39,485 questions was divided among the annotators for the first two cycles. Annotators independently reviewed and annotated their assigned questions without receiving guidance from others. In the second cycle, the same set of questions was reassigned to different annotators, ensuring each question was reviewed by two different annotators across the two cycles. In the third cycle, the main author independently annotated all 39,485 questions, serving as the third annotator for each question.

Our custom Java annotation application streamlined the annotation process. The tool incorporated validation checks to ensure annotations conformed to the established guidelines (see figures 2 and 3). The application provided an intuitive interface that enabled annotators to easily select categories, entity types, and answer spans, or mark questions as unanswerable. Furthermore, the application automated data management by automatically recording annotations alongside important metadata such as paragraph and sentence numbers. Here is a breakdown of the application workflow,

with numbered steps corresponding to the numbers in fig. 2 and fig. 3:

a) Annotating Answerable Questions (see fig. 2)

1. Upon launch, the only active button is "Open Zip." Clicking it allows the annotator to select an archive (a zip file) containing questions and corresponding articles. This action automatically opens the first question and its corresponding Urdu article within the application interface. The app automatically populated the *Document ID* field.
2. The annotator then selects the appropriate category (e.g., History, Science) from a predefined list for the question that best reflects the subject of the question.
3. The annotator then chooses the most relevant entity type (e.g., Person, Location) from a provided list.
4. The annotator reads the complete article to find the answer. Once found, the answer text is selected and copied. The app automatically populates *Paragraph No.*, and the *Sentence No.* fields from the current document with the correct annotations for the selected answer text.
5. The copied answer is then transferred into the dedicated *Answer* box.
6. Clicking *Save* stores the complete information consisting of assigned category, selected entity type, paragraph number, sentence number, and the answer text against the current question. If any information is missing, then this action notifies the user to complete the missing information.
7. The *Next* button allows the annotator to proceed to the next question if the information to the current question is successfully saved.

b) Annotating Unanswerable Questions (see fig. 3)

1. Repeat steps 1, 2, and 3 as given above.
4. The annotator reads the complete article to find the answer. If the answer cannot be found, the annotator then selects the *No Answer* button. Hence, the question is classified as an unanswerable question.
5. The above action automatically populates the *Answer* box with a *NULL* value. A value of 0 is also automatically populated in the *Paragraph No.* and the *Sentence No.* fields, signifying that the answer cannot be found in the current document.
6. Repeat steps 6, and 7 as given above.

The custom application comprehensively stores the annotated data, including the question, the answer text, or a *NULL* value if the answer is not found, and detailed metadata that captures the answer's association with the corresponding Urdu Wikipedia article. In addition, a Boolean *Answerable* field that stores the value **True** if the question is answerable, or a value **False** indicating the question is unanswerable is also stored against each question. We plan to utilize this field in future work.

¹¹<https://lahore.comsats.edu.pk/cs/> Last visited: 21-July-2022

FIGURE 2. Custom data annotation application with numbered Steps - Annotating answerable question.

FIGURE 3. Custom data annotation application with numbered Steps - Annotating unanswerable question.

4) ENSURING ANNOTATOR CONSENSUS

To ensure high-quality annotations, we implemented a rigorous process that prioritized consensus among annotators. **Unanimous agreement** from all three annotators was required for a question to be included in the final dataset. This included agreement on whether the question was answerable or unanswerable, and the selected answer span. Questions lacking unanimous agreement were excluded to maintain high annotation quality and reliability.

We employed a triple annotation approach to enhance the accuracy, reliability, and validity of our dataset. This approach offers several key advantages. It mitigates the influence of individual biases or errors, ensuring more reliable data. Unanimous agreement from all three annotators

is required, ensuring that only questions with clear and unambiguous responses are included. This approach also aligns with established best practices in NLP research, where multiple annotations and consensus-building are standard procedures for creating high-quality datasets [2], [4], [32], [33], [34]. Furthermore, with a reasonable number of trained annotators, we could effectively implement this rigorous approach without exceeding time or resource constraints.

5) ANNOTATION PROCESS OUTCOME

This meticulous annotation process resulted in the creation of UrFQuAD-v1, a high-quality Urdu ODFQA dataset comprising 3,985 questions, including 1,006 answerable and 2,979 unanswerable questions. These questions are linked to

TABLE 1. UrFQuAD-v1 corpus question domains.

Domain	Questions
General Knowledge	1,550
History	1,085
Sports	347
Entertainment	340
Science	274
Geography	179
Religion	117
Language	31
Economics	28
Computer	26
Arts	8
Total	3,985

TABLE 2. UrFQuAD-v1 corpus main statistics.

Total number of questions	3,985
Answerable questions	1,006
Unanswerable questions	2,979
Total number of articles in the corpus	584

584 Urdu Wikipedia articles. The rigorous consensus requirement, including the triple annotation approach, ensured the reliability and suitability of the dataset for developing and evaluating Urdu ODFQA systems. This dataset serves as a valuable resource for advancing ODFQA research in low-resource languages and supporting the development of robust Urdu ODFQA systems.

Table 1 shows that the question set of the UrFQuAD-v1 corpus belongs to a diverse set of domains and summarizes the number of questions in each subject category.

F. CORPUS CONSOLIDATION AND STANDARDIZATION

We divided the 3,985 questions (1,006 answerable, and 2,979 unanswerable) into the train and test sets using 80%-20% ratios. This resulted in a train set of 3,189 questions (805 answerable, 2,384 unanswerable), and a test set of 796 questions (201 answerable, 595 unanswerable). Table 2 details the key corpus statistics, while Table 3 details the train and test data split statistics.

To enhance the usability, we standardized both the UrFQuAD-v1 corpus train and test sets, including their corresponding Urdu Wikipedia articles into two formats: (1) NQ format,¹² as this mirrors the data format used during the manual annotation process, and (2) SQuAD-v2 format,¹³ as a convenience. The corpus data is available

¹²https://github.com/google-research-datasets/natural-questions/blob/master/toy_example.md Last visited: 21-July-2022

¹³https://huggingface.co/datasets/rajpurkar/squad_v2 Last visited: 21-July-2022

to download from this link¹⁴ under a Creative Commons Zero v1.0 Universal license,¹⁵ enabling open access for future research.

IV. BASELINE TECHNIQUES

To illustrate the potential of the proposed UrFQuAD-v1 corpus for the development, and evaluation of Urdu open-domain factoid question-answering systems, we have designed and applied all techniques as a two-stage question-answering pipeline. This pipeline is based on the *Retriever-Reader* [35] model (see Section IV-A). We have designed our techniques based on this model because of the following reasons: (1) This allows for the selection and the coupling of various retriever and reader models, and (2) This provided us the ability to evaluate and compare the various retriever and reader components individually.

For the comparison and benchmarking purposes, the Retrieval-Reader method was further adopted using three different designs: (1) Sparse-vector Retrieval with a Transformer Reader (see Sections IV-B, and IV-C), (2) Dense-Vector Retrieval with a Transformer Reader (see Sections IV-G, and IV-I), and (3) Retrieval-Augmented Generation (RAG) (see Section IV-J). We now describe the details of these methods.

A. RETRIEVAL-READER MODEL

The Retriever-Reader model is a question-answering (QA) architecture designed for large-scale datasets [35]. It uses a *Retriever* component to quickly narrow down relevant documents, and a *Reader* component to extract the answer span from these documents. The Retriever often employs techniques like BM25 [36], TF-IDF [37], or Dense Passage Retrieval (DPR) [26] for efficiency, while the Reader typically uses neural networks like BERT [20] for in-depth analysis, and identification of the precise answer span within the retrieved contexts.

We now describe the details of the Retrieval-Reader designs as used in our experiments. We will first describe the details of the retriever models, followed by the details of the respective reader models.

B. SPARSE-VECTOR RETRIEVAL

Sparse-vector retrieval represents documents and queries as high-dimensional vectors, primarily consisting of zeros. Using similarity calculations like cosine similarity, it ranks documents based on their relevance to the query, often relying on exact keyword matching. Methods like TF-IDF (Term Frequency-Inverse Document Frequency) [37] and BM25 [36] assign weights to terms, emphasizing significant ones for efficient retrieval, even with long documents. Combined with an inverted index, sparse-vector retrieval is computationally

¹⁴https://pern-my.sharepoint.com/:f:/g/personal/fa18-pcs-001_cuilahore_edu_pk/Epp7HUS0D4xBrkcKN0qzu7MBzCc1gV0Snh0er0qpbtHflw?e=hDmiSv

¹⁵<https://creativecommons.org/publicdomain/zero/1.0/deed.en> Last visited: 25-July-2022

TABLE 3. UrFQuAD-v1 corpus train and test split statistics.

	Train	Test
Total	3,189	796
Answerable questions	805	201
Unanswerable questions	2,384	595

TABLE 4. UrFQuAD-v1 Corpus Characteristics.

	Articles	Questions	Answers
Total	584	3,985	1,006
Total number of tokens	3,337,006	40,175	3,340
Average number of tokens	5,714.1 words	10 words	3 words
Maximum number of tokens	78,309 words	27 words	48 words
Minimum number of tokens	37 words	5 words	1 word

efficient for large-scale datasets. Its effectiveness has been demonstrated in various NLP tasks, including comparisons with dense retrievers for open-domain QA [38], domain-specific retrievals [39], and hybrid retrieval systems [40].

For our experiments, we selected the *BM25* as the baseline sparse-vector retriever, given its proven effectiveness [41]. Furthermore, we have used Elasticsearch¹⁶ (version 7.9.2) as the document store for the following reasons: (1) It facilitates efficient storage and management of the UrFQuAD-v1 dataset during experimentation, and (2) It provides a well-optimized *BM25* implementation. The Elasticsearch document store was coupled with its provided *BM25* retriever as outlined in the multi-passage BERT QA model [42]. To improve relevant passage identification, we applied document chunking, splitting documents into 200-word segments with a 10-word overlap. This setup leveraged the strengths of both *BM25* and Elasticsearch for efficient retrieval within our question-answering system. Finally, to assess real-world retrieval performance, we varied the number of retrieved documents per query using different retrieval thresholds ($k = [1, 3, 5, 10, 20]$). By varying the retrieval threshold, we evaluated the impact of the number of retrieved documents on both the retriever's performance and the overall system's accuracy.

C. TRANSFORMER READERS FOR THE SPARSE-VECTOR RETRIEVAL

To select the suitable Reader models for the Urdu ODFQA task, we focused on Transformer models having the following properties: (1) Must be multilingual, i.e., have been trained on languages beyond English, (2) Have Urdu language support, and trained on substantial Urdu data, and (3) Have

computational cost within our budget (see Section V). The *XLNet* – *RoBERTa* – *Large* [22], *XLNet* – *RoBERTa* – *Large* – *Squad2*¹⁷ [22], and *BERT* – *Base* – *Multilingual* – *Cased*¹⁸ [20] were selected as promising candidates due to their multilingual nature, Urdu language support, and their ability to be fine-tuned on Urdu question answering tasks using the UrFQuAD-v1 corpus. These models serve as strong baselines and benchmarks for the Urdu ODFQA task. The details of these three models are as below.

D. BERT-BASE-MULTILINGUAL-CASED

The *BERT* – *Base* – *Multilingual* – *Cased* is a multipurpose language model capable of understanding and processing multiple languages. Built on the transformer architecture [20] with an encoder-only variation, it comprises 12 layers with 768 hidden units each and containing 12 attention heads. Its total input length (context length) is limited to 512 tokens. This model consists of approximately 110 million parameters and has been pre-trained on a large dataset of 104 languages using a shared vocabulary.

Also, unlike its case-insensitive counterpart, this model preserves case information (Cased-version), which can be beneficial for certain NLP tasks. Its bidirectional context ability allows it to process text bidirectionally and allows it to capture richer contextual information. Furthermore, its pre-training objectives consist of including Masked Language Modelling (MLM) and Next Sentence Prediction (NSP). MLM involves predicting masked tokens based on the context, while NSP predicts the relationship between

¹⁷<https://huggingface.co/deepset/xlnet-roberta-large-squad2> Last visited: 21-July-2022

¹⁸<https://github.com/google-research/bert/blob/master/multilingual.md> Last visited: 21-July-2022

¹⁶<https://www.elastic.co/elasticsearch> Last visited: 21-July-2022

sentence pairs. These pre-training objectives thus equip this model to excel in various natural language processing tasks, such as question-answering [5], [20], text classification [43], and named-entity recognition [44].

E. XLM-ROBERTA-LARGE

The *XLM – RoBERTa – Large* is a multilingual Transformer model designed for cross-lingual NLP tasks. Built upon the RoBERTa [45] architecture, it comprises 24 layers with 1,024 hidden units and 16 attention heads, totaling approximately 550 million parameters. The XLM-RoBERTa-Large model has a maximum context length of 512 tokens. Trained on a large multilingual corpus, the model leverages Masked Language Modelling to acquire a deep understanding of language patterns across multiple languages. This cross-lingual pre-training enables XLM-RoBERTa-Large to excel in various NLP tasks, such as machine translation [46], sentiment analysis [47], and question answering [22], by effectively capturing semantic and syntactic similarities between languages.

The *BERT – Base – Multilingual – Cased* model, while including multiple languages, is primarily trained on a corpus focused on the top languages with the largest Wikipedia article collections [20]. This might limit its understanding of low resource languages like Urdu. In contrast, the *XLM – RoBERTa – Large* model, is specifically designed for cross-lingual tasks, and is trained on a dataset encompassing 100 languages [22] using a shared vocabulary. This broader training makes *XLM – RoBERTa – Large* a stronger candidate for tasks involving Urdu or other low-resourced languages.

F. XLM-ROBERTA-LARGE-SQUAD2

The *XLM – RoBERTa – Large – Squad2* model shares the same architecture as *XLM – RoBERTa – Large*, but it has been fine-tuned primarily to improve performance on the extractive question-answering task. This training involved the full SQuAD-v2 dataset [5], which focuses on finding answers directly within the provided text. Also, SQuAD-v2 includes questions that do not have an answer in the provided context, so the model learns to not only find answers but also to recognize when a question cannot be answered based on the context.

We selected this model to see how well it performs on our Urdu factoid question answering task without any additional finetuning (zero-shot setting). We treated this setting as the baseline for our experiments. This allows us to compare its transfer learning capabilities to the other two models (*BERT – Base – Multilingual – Cased*, and *XLM – RoBERTa – Large*), and provides a benchmark for their performance in zero-shot settings (see Section VI). Also, for a direct comparison of model performance in the fine-tuned scenarios, all three transformers underwent finetuning on the entire UrFQuAD-v1 development set (see Section V for the specific finetuning configuration details).

G. DENSE-VECTOR RETRIEVAL WITH A TRANSFORMER READER

To evaluate the various retrieval methods for the Urdu ODFQA, we compared classical techniques (non-learning based, detailed in Section IV-B) with a neural-based retrieval system (learning-based) known as *Dense-Vector* retrieval. We will now detail the Dense-Vector retrieval, and the Reader models used in our experiments.

H. DENSE-VECTOR RETRIEVAL

Dense-vector retrieval employs high-dimensional, continuous vector representations, called embeddings, to capture semantic similarities between documents and queries. Unlike sparse-vector retrieval, which relies on exact keyword matching, dense-vector retrieval can retrieve documents that are semantically similar, even if they lack exact keyword overlap. This advantage also enables dense-vector retrieval to manage out-of-vocabulary words, i.e., words not encountered during training.

For Dense-Vector retrieval, we specifically use the pre-trained *distiluse – base – multilingual – cased – v2*¹⁹ [48] Sentence-BERT [49] model. This model utilizes Siamese BERT-Networks to generate semantically rich sentence embeddings and includes Urdu language data. Sentence-BERT builds upon BERT but is optimized for sentence-level tasks. It employs Siamese architecture to learn sentence embeddings that are similar for semantically equivalent sentences and dissimilar for others. By applying pooling operations on token embeddings produced by BERT, the model derives fixed-size sentence embeddings. The effectiveness of Sentence-BERT in tasks like semantic textual similarity, question answering, and information retrieval makes it a suitable choice for our Urdu QA system [49].

The Sentence-BERT model is an improvement of BERT and is optimized for sentence-level tasks. It employs Siamese architecture to learn sentence embeddings that maximize similarity between semantically equivalent sentences while minimizing distance between dissimilar ones. The model derives fixed-size sentence embeddings through pooling operations on token embeddings produced by BERT. The model has been demonstrated to be effective in tasks, such as semantic textual similarity, question answering, and information retrieval. This makes the Sentence-BERT a suitable choice for our task. Furthermore, the *distilbert – base – multilingual – cased – v2* model, being a distilled version of BERT, benefits from knowledge distillation, allowing for efficiency without sacrificing performance. Its multilingual training, including Urdu data, makes it well-suited for our Urdu ODFQA task.

¹⁹<https://huggingface.co/sentence-transformers/distiluse-base-multilingual-cased-v2> Last visited: 21-July-2022

I. TRANSFORMER READERS FOR THE DENSE-VECTOR RETRIEVAL

For the Reader component, we selected the same *XLM – RoBERTa – Large – Squad2* (see Section IV-F), *XLM – RoBERTa – Large* (see Section IV-E), and *BERT – Base – Multilingual – Cased* (see Section IV-D) models.

J. RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation (RAG) [50] is a two-stage pipeline, similar to the Retriever-Reader model. It combines a Retriever component with a Large Language Model (LLM) Generator. RAG enhances LLMs by incorporating external knowledge during the generation process. This supplements the LLM's existing knowledge (pretrained data), leading to more accurate and informative outputs. RAG's effectiveness in retrieving relevant information from external sources has been well demonstrated in tasks like question answering [50], where it achieved state-of-the-art performance on benchmark corpora, like SQuAD-v2 [5] and Natural Questions [2], highlighting its ability to accurately answer factual questions using retrieved knowledge.

We now describe the retriever-generator components for the RAG model as selected in our Urdu ODFQA task.

K. RAG RETRIEVAL COMPONENT

We extended the RAG model by Lewis et al. [50] using BM25 for the retrieval. Our experiments revealed that BM25 outperformed Dense-Vector retrieval, as detailed in Section V.

L. RAG GENERATOR COMPONENT

OpenAI's Generative Pre-trained Transformer (GPT) models have significantly advanced the field of natural language processing. The GPT-3.5 series, trained on massive multilingual datasets [27], demonstrated state-of-the-art performance across various NLP tasks, including creative text generation, translation, code generation, and question answering [51]. Subsequent iterations like GPT-4 [52], and its variants, GPT-4o,²⁰ and GPT-4o-mini,²¹ have further enhanced these capabilities, incorporating multimodal functionalities and achieving even greater understanding, generation, and contextual coherence.

Given the impressive performance in various NLP domains [27], [51], we selected the GPT-3.5-Turbo-0125 and GPT-4o-mini-2024-07-18 models as the Reader components for our Urdu ODFQA task. These model's ability to handle long dependencies within text, strong zero-shot transfer learning abilities, and the potential for finetuning on the UrFQuAD-v1 corpus, make them robust baselines for our Urdu ODFQA task.

²⁰<https://openai.com/index/hello-gpt-4o/> Last visited: 13-May-2024

²¹<https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/> Last visited: 18-July-2024

V. EXPERIMENTAL SETUP

This section delves into the key components of our system. We will discuss the dataset used, explore state-of-the-art Transformer and Large Language Model approaches, and detail our performance evaluation methodology.

A. DATASET

For our experiments, we used the entire UrFQuAD-v1 corpus, comprising 3,189 training and 796 test question sets (see Table 3). All experiments included both answerable and unanswerable questions. UrFQuAD-v1 contains Urdu questions, their corresponding articles, and annotated answers. In experiments using the three transformer models (see Sections IV-D, IV-E, and IV-F), we used the complete training set (3,189 questions) for finetuning models, and the complete test set (796 questions) for the zero-shot inference. The GPT-4o-mini-2024-07-18 model was finetuned on a training set consisting of 1,610 questions, including 805 answerable and 805 unanswerable questions (see Section V-B). In both zero-shot and finetuned inference settings, the full test set of 796 questions were used for both models (see Section V-B). Detailed corpus statistics can be found in Section III.

B. TECHNIQUES

To comprehensively evaluate different retrieval and reader model combinations, we conducted a total of twelve experiments. Six experiments explored sparse-vector retrieval (see Sections IV-B), using both zero-shot, and fine-tuned transformers (see Sections IV-D, IV-E, and IV-F). Three experiments investigated dense-vector retrieval (see Sections IV-G) coupled with fine-tuned transformers (see Section IV-I). Finally, we examined the performance of the Retrieval-Augmented Generation (RAG) method (see Section IV-J) using two zero-shot, and one finetuned inference experiment (see Section IV-L). We now describe the configuration details of these experiments.

All experiments were performed on the Google Colab Pro²² environment equipped with dual Intel Xeon vCPUs (16 cores), 83 GB of System RAM, and a single NVIDIA A100 GPU having 40 GB VRAM. All code was written in Python²³ version 3.10 and executed using the PyTorch framework.²⁴

To maintain consistency and optimize performance, we adhered to the recommended transformer model (see Sections IV-D, IV-E, IV-F) hyperparameter settings for both finetuning and zero-shot inference. A batch size of 16 samples was used across all models to efficiently utilize the GPU's 40 GB VRAM. Additionally, default tokenizers were used to minimize configuration overhead. For efficient retrieval, Wikipedia articles were segmented as described in Section IV-B. Document embeddings were

²²<https://colab.research.google.com/signup> Last visited: 21-July-2022

²³<https://www.python.org/> Last visited: 21-July-2022

²⁴<https://pytorch.org/> Last visited: 21-July-2022

standardized to 768 dimensions, aligning with each model's recommendation.

To optimize the training efficiency and prevent overfitting, we used the AdamW [53] optimizer with a fixed learning rate of $1e - 5$, and weight decay of 0.01. For GPU memory optimization, we configured the gradient accumulation step to 1, to accumulate gradients for one batch update at a time. Additionally, we used a warmup proportion of 0.2, which gradually increases the learning rate during the initial training phase. To handle the length of input sequences, questions were capped at 27 tokens, answers at 100 tokens, and the overall input at 256 tokens (see Table 4). The number of training epochs was set between 1 and 10.

Furthermore, to ensure compatibility with the transformer models used in this study having a maximum input length of 512 tokens, and efficiently utilize the larger context windows of LLMs (GPT-3.5-Turbo-0125 with 4,096 tokens and GPT-4o-mini with 128,000 tokens), we implemented input length limits. These limits ensured efficient utilization of computational resources, and reduced memory consumption and processing time. These limits were carefully chosen: 27 tokens for questions, 48 tokens for the answers, 100 tokens for answers, and 256 tokens for the overall input (see Table 4). These values were determined based on the observed token distributions in the dataset and the computational requirements of the models. By setting these limits, we ensured that all corpus samples could be processed without truncation while maintaining compatibility with both transformer models and LLMs. We also split documents into 200-word segments to efficiently process longer passages while adhering to the input length requirements (see section IV-B).

As stated earlier, our experiments utilized an NVIDIA A100 GPU with 40GB of VRAM, enabling full finetuning of large transformer models like BERT and XLM-RoBERTa-Large without encountering memory or processing limitations. We opted for full finetuning for all models to maximize model performance by allowing all parameters to be adjusted. This approach ensured consistency across all models in our study, facilitating fair comparisons and efficient analysis.

Our RAG experiments followed a two-stage process. First, BM25 identified relevant documents from the UrFQuAD-v1 dataset. Next, the GPT-3.5-Turbo-0125, and the GPT-4o-mini-2024-07-18 models were used for zero-shot inference on the full test set. Afterward, the GPT-4o-mini-2024-07-18 was finetuned on a train subset, and the finetuned model was then used for inference on the full test set. To optimize the retrieval performance, a sliding window approach was applied to documents (see Section IV-B). To ensure focused and factual responses, we set the *max_tokens* parameter to 100 tokens for fixing the output answer's length, and *n* to 1 for returning a single answer per prompt (question).

During the finetuning process for the GPT-4o-mini model, we maintained the default hyperparameters provided by the OpenAI API. However, we set the temperature to 0 to ensure fully deterministic outputs, with the number of epochs set to 3. To align the model with our UrFQuAD-v1 corpus,

we employed instruction finetuning via OpenAI's API, adhering to their recommended data formatting guidelines.²⁵ This approach involved careful data preparation, and following the correct formatting guidelines.

The finetuning dataset consisted of 3,189 questions, with 805 being answerable and 2,384 being unanswerable. However, due to cost constraints, we created a balanced training subset of 1,610 examples. This subset included all 805 answerable questions and an equal number (805) of unanswerable questions from the original training set. This balanced approach aimed to optimize resource utilization while mitigating the risk of overfitting. By preventing the model from developing a bias towards consistently providing answers or refraining from providing answers, we aimed to enhance its ability to accurately distinguish between answerable and unanswerable queries.

Each finetuning example was formatted as a JSON object representing a conversation, that included several components: the System Prompt, which guided the model's behavior, instructing it to find the answer within the context, or respond with an empty string if not found. Next, the User Message, which provided the context and question in Urdu. Similarly, the Assistant Response, which provided the ground truth answer or an empty string. Next, the Context, which included the Urdu passage from which an answer could be derived. Finally, the Question, which provided the Urdu question related to the context. Figures 4 and 5 illustrate examples of formatted finetuning training samples for answerable and unanswerable questions.

Furthermore, we could not utilize Parameter-Efficient Fine-Tuning (PEFT) techniques, as OpenAI models can only be finetuned via OpenAI's API. In general, PEFT techniques require access to the model's internal architecture and the ability to modify or add parameters—capabilities that are currently not supported through the OpenAI finetuning API.

C. EVALUATION METHODOLOGY

The Urdu open-domain factoid question-answering task was treated as a supervised learning problem, more specifically an extractive question-answering problem. The retrieval model takes as input an Urdu question and the Urdu Wikipedia document repository, producing a set of candidate passages as output. Subsequently, the reader model processes the question and retrieved passages to generate an Urdu answer or indicate unanswerability. For effective evaluation of all models, the entire UrFQuAD-v1 corpus was split into train (3,189 questions) and test (796 questions) sets. Due to cost considerations, the GPT-4o-mini-2024-07-18 model was fine-tuned using 1,610 samples coupled with a retrieval threshold of $k = 1$ from the train set, balanced across answerable and unanswerable questions.

Since all our experiments employed a retriever-reader architecture, we evaluated both the retrieval and reader

²⁵<https://platform.openai.com/docs/guides/fine-tuning/> Last visited: 15-Dec-2024

```

"messages": [
  {
    "role": "system",
    "content": "You are a helpful QA assistant. Given a context and a question, return the short factual answer. If the context does not contain the answer, return an empty string."
  },
  {
    "role": "user",
    "content": "Context: نیو یارک شہر (یعنی نیا یارک) ریاستہائے متحدہ امریکا کا سب سے بڑا شہر ہے اور دنیا کے بھی کہا جاتا ہے۔ ریاست نیویارک کے اس شہر کی (Big Apple) عظیم ترین شہروں میں سے شمار ہوتا ہے۔ اس کو بگ ایپل نیو یارک شہر کو کس نام سے بھی جانا جاتا ہے؟\n\nQuestion: 8.2 ملین بے جبکہ شہر 321 مربع میل پر پھیلا ہوا ہے۔ ہے؟"
  },
  {
    "role": "assistant",
    "content": "بگ ایپل"
  }
]

```

FIGURE 4. OpenAI instruction finetuning format Example - Answerable question.

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful QA assistant. Given a context and a question, return the short factual answer. If the context does not contain the answer, return an empty string!"
    },
    {
      "role": "user",
      "content": "Context: پاکستان کے دار الحکومت اسلام آباد کو 1960 کی دہائی میں تعمیر کیا گیا تھا۔ یہ ملک کا سیاسی اور سفارتخانے یہاں واقع ہیں۔\n\nQuestion: اسلام آباد کی آبادی کتنی ہے؟"
    },
    {
      "role": "assistant",
      "content": ""
    }
  ]
}
```

FIGURE 5. OpenAI instruction finetuning format Example - Unanswerable question.

models individually using appropriate metrics. Retrieval models (sparse-vector and dense-vector) were assessed using the Recall (eq. 1), Precision (eq. 2), Mean Reciprocal Rank (MRR) (eq. 3), and Mean Average Precision (MAP) (eq. 4) metrics, with retrieval thresholds set at $k = 1, 3, 5, 10, 15, 20$, to measure their effectiveness in finding relevant passages. The Reader models were evaluated using the Exact Match (EM) (eq. 5) and F_1 (eq. 6) metrics against the gold standard answers. This assessment helps us gauge their ability to accurately generate answers that precisely match the ground truth information retrieved by the passages.

In the below given equations, *Relevant Passages* denotes the number of passages in the document collection that contain information related to the answer to the user’s question, and *Retrieved Passages* denote the number of passages that the retrieval model identifies and presents to the reader model as potentially containing the answer. It’s important to note that the retrieved passages might also include irrelevant ones that do not actually contain the answer. Furthermore, the Mean Reciprocal Rank calculates the average of how quickly the retriever finds the first relevant passage for each question, and the Mean Average

Precision measures the average quality of a retrieval model's ranked list of passages across all queries, considering both precision and recall at various ranks. For the reader models, Exact Match measures the proportion of times the generated answer precisely matches the gold standard answer (human-annotated answer), and F_1 is the weighted harmonic mean of the Precision and Recall.

RETRIEVAL

$$Recall = \frac{\text{Relevant Passages}}{\text{Total Relevant Passages in Collection}} \quad (1)$$

$$Precision = \frac{\text{Relevant Passages}}{\text{Total Retrieved Passages}} \quad (2)$$

$$MRR = \frac{1}{n} \sum_{i=1}^n \frac{1}{\text{rank}_i} \quad (3)$$

where, n is the total number of questions, and rank_i is the rank position of the first relevant passage retrieved for the question i (1 indicates the first position, 2 indicates the second, and so on).

$$MAP = \frac{1}{n} \sum_{i=1}^n \text{Average Precision}(i) \quad (4)$$

where, n is the total number of questions, and the Average Precision for the question i is calculated as the area under the precision-recall curve (PRC) for that specific query.

READER

$$EM = \frac{\text{Predictions Matching Gold Standard Answers}}{\text{Total Predictions}} \quad (5)$$

$$F_1 = \frac{2(\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}} \quad (6)$$

VI. RESULTS AND ANALYSIS

Tables 5-8 present the retrieval and reader model performance on the UrFQuAD-v1 corpus. We evaluate the BM25 and Sentence-BERT retrieval at different thresholds ($k = 1, 3, 5, 10, 15, 20$) and compare the three Transformer Readers (BERT-Base-Multilingual-Cased, XLM-RoBERTa-Large, and XLM-RoBERTa-Large-Squad2) in both zero-shot and fine-tuned settings with both retrievers. Additionally, we explore the performance of two LLM Generators (GPT-3.5-Turbo-0125 and GPT-4o-mini-2024-07-18) in zero-shot and the finetuned GPT-4o-mini-2024-07-18 with BM25 retrieval. The complete results can be found at this link.²⁶

We begin by discussing the importance and the benefits of setting different retrieval thresholds, as observed in Table 5. Our experiments revealed that increasing the value of k , the number of top documents retrieved, leads to improved performance in both the retrieval and reader components of our question-answering system. This improvement can be attributed to several factors, namely as k increases, the retrieval model returns a larger set of documents (see Table 5), which increases the likelihood of including relevant

documents that contain the correct answer. This is particularly important because Retrieval models may not always rank the most relevant documents at the top due to limitations in understanding the query or document content variations. Also, a higher k determines a larger range of documents that may contain unique descriptions of the relevant information. With a more exhaustive set of documents (see Tables 6, 8), the reader model has better chances of locating and extracting the correct answer. This is made possible as more documents provide richer context, which can aid the reader in understanding and answering the question accurately.

In addition, different documents may describe the relevant information differently; having multiple sources increases the chance of matching the query's meaning. While a higher k may introduce additional irrelevant documents, the reader model is configured to filter through the irrelevant information and focus on extracting the correct answer. This results in the increase in the recall, which often leads to a positive effect on performance metrics, like F_1 and EM scores. Finally, our results clearly demonstrate a trend where both the retrieval and the reader performance improve with increasing k . For instance, when k is increased from 1 to 20, we observed an improvement in the *Recall* score of the BM25 retriever from 0.31 to 0.81 (see Table 5), and the corresponding F_1 score of the finetuned *XLM – RoBERTa – Large – Squad2* model from 0.03 to 0.61 (see Table 6).

Additionally, it's important to consider the trade-off between the performance and the computational cost when selecting the retrieval threshold (k). Increasing k can improve performance but also increases computational costs.

As Table 5 demonstrates, BM25 significantly outperforms the Sentence-BERT (distiluse-base-multilingual-cased-v2) across all retrieval thresholds ($k = 1, 3, 5, 10, 15, 20$) with scores of ($\text{Recall}@1 = 0.31$, $\text{Recall}@3 = 0.56$, $\text{Recall}@5 = 0.64$, $\text{Recall}@10 = 0.74$, $\text{Recall}@15 = 0.78$, and $\text{Recall}@20 = 0.81$) on the UrFQuAD-v1 corpus. This establishes BM25 as the preferred retriever for our Urdu open-domain factoid question-answering task, aligning with existing research establishing it as a robust baseline in information retrieval [41], [54]. BM25's effectiveness stems from its probabilistic model, which accurately captures term relevance, and its document length normalization, crucial for handling Urdu languages' variable document lengths. Additionally, BM25's reliance on TF-IDF heuristics leverages direct term frequency and document frequency statistics, often surpassing the effectiveness of semantic embeddings in text corpora. While Sentence-BERT is a multilingual model fine-tuned on Urdu data, its performance can be limited by insufficient domain-specific finetuning. Multilingual models may struggle to capture language-specific nuances or domain-specific terms unless extensively fine-tuned on highly relevant data. Furthermore, Sentence-BERT's performance can be influenced by the quality of pre-trained models, and their adaptability to specific languages like Urdu [48], [49].

²⁶https://pern-my.sharepoint.com/:f:/g/personal/fa18-pcs-001_cuilahore_edu_pk/Epp7HUS0D4xBrkcKN0qzu7MBzCc1gV0Snh0er0qpbthHfW?e=hDmiSv

TABLE 5. Retrieval performance of the BM25 and the Sentence-BERT (distiluse-base-multilingual-cased-v2) on the UrFQuAD-v1 corpus.

	Retrieval Performance							
	BM25				Sentence-BERT			
	RM ¹	MRR ²	P ³	MAP ⁴	RM ¹	MRR ²	P ³	MAP ⁴
k = 20	0.81	0.46	0.10	0.40	0.59	0.24	0.07	0.21
k = 15	0.78	0.46	0.12	0.41	0.53	0.24	0.07	0.21
k = 10	0.74	0.45	0.14	0.42	0.46	0.23	0.08	0.21
k = 5	0.64	0.44	0.20	0.43	0.37	0.22	0.11	0.21
k = 3	0.56	0.42	0.24	0.42	0.27	0.20	0.11	0.20
k = 1	0.31	0.31	0.31	0.31	0.14	0.14	0.14	0.14

¹ Recall@Multiple documents² Mean Reciprocal Rank³ Precision⁴ Mean Average Precision**TABLE 6.** Zero-shot and finetuned transformers performance with the BM25 retriever on the UrFQuAD-v1 corpus.

	BM25 Retrieval + Transformer Readers											
	Zero-Shot						Finetuned					
	BERT ¹		XLM-RL ²		XLM-RL-S ³		BERT ¹		XLM-RL ²		XLM-RL-S ³	
	F_1	EM	F_1	EM	F_1	EM	F_1	EM	F_1	EM	F_1	EM
k = 20	0.07	0.00	0.08	0.00	0.54	0.39	0.35	0.23	0.58	0.38	0.61	0.41
k = 15	0.07	0.00	0.07	0.00	0.52	0.38	0.34	0.22	0.55	0.37	0.58	0.39
k = 10	0.05	0.00	0.05	0.00	0.49	0.35	0.27	0.17	0.50	0.32	0.54	0.36
k = 5	0.04	0.00	0.04	0.00	0.41	0.30	0.19	0.11	0.39	0.26	0.44	0.31
k = 3	0.04	0.00	0.03	0.00	0.31	0.22	0.13	0.07	0.30	0.20	0.34	0.24
k = 1	0.00	0.00	0.00	0.00	0.08	0.06	0.00	0.00	0.00	0.00	0.03	0.03

¹ BERT-Base-Multilingual-Cased² XLM-RoBERTa-Large³ XLM-RoBERTa-Large-Squad2

Table 6 demonstrates that the XLM-RoBERTa-Large-Squad2 model, paired with BM25 retrieval, outperforms other Transformer Readers in both zero-shot and finetuned settings. In zero-shot settings, it achieves scores of ($F_1 = 0.08@k = 1$, $F_1 = 0.31@k = 3$, $F_1 = 0.41@k = 5$, $F_1 = 0.49@k = 10$, $F_1 = 0.52@k = 15$, $F_1 = 0.54@k = 20$). finetuning further enhances its performance, yielding scores of ($F_1 = 0.03@k = 1$, $F_1 = 0.34@k = 3$, $F_1 = 0.44@k = 5$, $F_1 = 0.54@k = 10$, $F_1 = 0.58@k = 15$, $F_1 = 0.61@k = 20$). These results establish the XLM-RoBERTa-Large-Squad2 as the best Transformer model for our Urdu open-domain factoid question-answering task.

XLM-RoBERTa-Large outperforms BERT-Base-Multilingual-Cased due to its significantly larger and more diverse multilingual training data. Thus, XLM-RoBERTa-Large generalizes better across languages and tasks [22]. Both models use masked language modeling (MLM), but XLM-RoBERTa-Large model's more diverse data sources enhance its performance. Unlike BERT-Base-Multilingual-Cased model's next sentence prediction (NSP), XLM-RoBERTa-Large relies solely on MLM, a more effective pre-training

objective for multilingual models [45]. Furthermore, XLM-RoBERTa-Large model's training data was filtered more rigorously, removing noisy text and improving data quality. Trained on 100 languages, XLM-RoBERTa-Large benefits from broader, more curated multilingual data, leading to better cross-lingual performance [22]. Additionally, the XLM-RoBERTa-Large model's architecture, optimized with larger batches, longer training, and more data, provides improved training stability and robustness, resulting in refined representation learning compared to BERT-Base-Multilingual-Cased [45]. XLM-RoBERTa-Large-Squad2's finetuning on the SQuAD-v2 dataset, which includes unanswerable questions, equips it to handle challenging question-answering scenarios effectively. This finetuning enables the model to learn specific patterns and details, significantly enhancing its performance in related tasks [5], [22]. Building upon the robust foundation of XLM-RoBERTa-Large, finetuning on SQuAD-v2 results in a model that excels at question-answering. This adaptation not only improves the model's ability to handle various question types but also enhances its generalizability within the domain

TABLE 7. Zero-shot and finetuned LLMs performance with the BM25 retriever @ $k=1$ on the UrFQuAD-v1 corpus.

BM25 + LLM Generators						
Zero-Shot				Finetuned		
GPT-3.5 ¹		GPT-4o-mini ²		GPT-4o-mini ³		
F_1	EM	F_1	EM	F_1	EM	
$k = 1$	0.21	0.18	0.47	0.45	0.81	0.81

¹ GPT-3.5-Turbo-0125

² GPT-4o-mini-2024-07-18

³ GPT-4o-mini-2024-07-18 Finetuned on 1,610 Training Samples @ $k = 1$

of reading comprehension. XLM-RoBERTa-Large-Squad2's specialized training allows it to outperform models without such finetuning.

Table 7 demonstrates that GPT-4o-mini-2024-07-18 model outperforms the GPT-3.5-Turbo-0125 model in zero-shot settings, achieving an F_1 score of 0.47 @ $k = 1$. Due to cost constraints, we evaluated both models at $k = 1$ threshold only. The GPT-3.5-Turbo-0125 model exhibits relatively lower F_1 and EM scores, typical for models in zero-shot scenarios relying solely on general pre-training. GPT-4o-mini-2024-07-18 model's superior performance reflects its advanced architecture, which enables better context handling, reasoning, and overall NLP capabilities [52]. Trained on a more extensive and diverse dataset compared to GPT-3.5-Turbo-0125, GPT-4o-mini-2024-07-18 achieves better generalization and performance across various tasks [52]. Additionally, optimized training procedures contribute to GPT-4o-mini's enhanced performance [52].

Table 7 indicates that the finetuned GPT-4o-mini-2024-07-18 model achieved the best performance among all models evaluated in this study, attaining an F_1 and EM score of **0.81**. We attribute several factors that have likely contributed to this performance. Firstly, finetuning GPT-4o-mini on the UrFQuAD-v1 dataset provides essential exposure to the dataset's specific language, terminology, and question-answer patterns. This direct experience allows the model to develop a deeper understanding of our QA problem, improving its ability to accurately extract answers and formulate responses compared to zero-shot learning. Specifically, finetuning enhances the model's knowledge by aligning its internal representations with the way information is structured and presented within the UrFQuAD-v1 corpus, minimizing the generation of irrelevant responses. Secondly, GPT-4o-mini benefits from a more advanced architecture compared to GPT-3.5-turbo-0125, enabling it to excel in complex reasoning tasks that require multiple steps or the integration of information from different sources. This improved architecture also allows for more precise utilization of passages retrieved by the BM25 algorithm, enabling the model to effectively leverage external knowledge to answer questions more accurately. The combination of targeted

TABLE 8. Finetuned transformers performance with the Sentence-BERT (distiluse-base-multilingual-cased-v2) retriever on the UrFQuAD-v1 corpus.

S-BERT Retrieval + Finetuned Transformer Readers						
	BERT ¹		XLM-RL ²		XLM-RL-S ³	
	F_1	EM	F_1	EM	F_1	EM
$k = 20$	0.26	0.16	0.41	0.25	0.44	0.28
$k = 15$	0.23	0.14	0.37	0.24	0.40	0.26
$k = 10$	0.20	0.13	0.33	0.21	0.36	0.23
$k = 5$	0.14	0.08	0.24	0.14	0.27	0.16
$k = 3$	0.07	0.03	0.15	0.09	0.17	0.10
$k = 1$	0.00	0.00	0.00	0.00	0.02	0.01

¹ BERT-Base-Multilingual-Cased

² XLM-RoBERTa-Large

³ XLM-RoBERTa-Large-Squad2

finetuning on UrFQuAD-v1 corpus, which optimizes the model for the specific task and data distribution, and the superior architectural capabilities of GPT-4o-mini, particularly in multi-step reasoning and information retrieval, explains its outperformance compared to both its zero-shot variant and the less advanced GPT-3.5-turbo-0125.

Another key observation from the Table 7 is the superior zero-shot performance of both LLMs compared to all Transformer models (see Table 6) @ $k = 1$. This indicates that the LLMs' architectural enhancements enable stronger generalization capabilities. Given these results, we can confidently predict that both LLMs will outperform the three Transformer models across all retrieval thresholds ($k = 1, 3, 5, 10, 15, 20$), with GPT-4o-mini-2024-07-18 emerging as the leading model on our Urdu ODFQA task.

Table 8 reveals that all Transformer Readers paired with Sentence-BERT retrieval underperform as compared to their performance with BM25 retrieval (see Table 6). This suggests that Sentence-BERT's inferior retrieval capabilities are the primary limiting factor in these configurations.

The resulting F_1 and EM scores underscore the inherent challenges of the Urdu open-domain factoid question-answering task. Despite these initial results, this research offers a promising baseline for researchers to compare their techniques for the Urdu open-domain factoid question-answering task on our proposed UrFQuAD-v1 corpus.

Further research could focus on enhancing the performance of Transformer models and LLMs through hyperparameter optimization and feature engineering. Addressing the challenges of limited training data for LLMs involves increasing the size and diversity of finetuning datasets, while considering the computational costs associated with large-scale training. Additionally, investigating language-specific nuances in the tokenization schemes, domain-specific embeddings, and ensuring consistent access to GPU resources can further improve model performance. By addressing these areas, researchers can contribute to important advancements in the Urdu open-domain factoid question-answering task.

VII. CONCLUSION

This paper introduces UrFQuAD-v1, a benchmark corpus for the Urdu open-domain factoid question-answering task. Comprising 3,985 questions (1,006 answerable, 2,979 unanswerable), UrFQuAD-v1 corpus covers diverse domains and entities. To evaluate Urdu question-answering systems, we employed a Retriever-Reader framework in three configurations: (1) Sparse-vector retrieval with Transformer Readers, (2) Dense-vector retrieval with Transformer Readers, and (3) Retrieval-Augmented Generation (RAG).

We selected the BM25 and the Sentence-BERT as our sparse-vector and dense-vector retrievers, respectively. These were paired with three state-of-the-art multilingual Transformer Readers: BERT-Base-Multilingual-Cased, XLM-RoBERTa-Large, and XLM-RoBERTa-Large-Squad2. Models were evaluated in both zero-shot and fine-tuned settings with BM25, and fine-tuned settings with Sentence-BERT. Additionally, we evaluated the GPT-3.5-Turbo-0125, and GPT-4o-mini-2024-07-18 LLMs in zero-shot and fine-tuned settings (GPT-3.5-Turbo-0125 only) with the BM25 retriever.

XLM-RoBERTa-Large-Squad2 paired with BM25 consistently outperformed other transformer models, achieving F_1 scores of ($F_1 = 0.08@k = 1$, $F_1 = 0.31@k = 3$, $F_1 = 0.41@k = 5$, $F_1 = 0.49@k = 10$, $F_1 = 0.52@k = 15$, $F_1 = 0.54@k = 20$) in zero-shot, and ($F_1 = 0.03@k = 1$, $F_1 = 0.34@k = 3$, $F_1 = 0.44@k = 5$, $F_1 = 0.54@k = 10$, $F_1 = 0.58@k = 15$, $F_1 = 0.61@k = 20$) in fine-tuned settings. Notably, GPT-4o-mini-2024-07-18 emerged as the top-performing model overall, achieving the highest zero-shot score ($F_1 = 0.47@k = 1$) and the best performance in the finetuned setting ($F_1 = 0.81@k = 1$).

In future work, we intend to conduct a comparative analysis of large language models (LLMs) on our UrFQuAD-v1 Urdu QA dataset, focusing specifically on their reasoning abilities. This will involve evaluating models such as OpenAI's o1, o3 and Deepseek R1. We are particularly interested in contrasting the computational approaches of these reasoning models, such as the brute-force computation often associated with large language models like those from OpenAI versus the algorithmic efficiency emphasized by the Deepseek R1 model.

Our evaluation would include standard metrics such as accuracy, as well as a qualitative analysis of the models' reasoning processes, examining the steps they take to answer complex questions. This will provide insights into how each model handles the linguistic and cultural nuances of Urdu, including their ability to resolve ambiguities, understand idiomatic expressions, and incorporate cultural context. Furthermore, we also plan to explore techniques like chain-of-thought prompting and hybrid methods that combine rule-based systems with neural architectures to enhance performance on the ODFQA task in Urdu.

We also intend to examine language-specific complexities in the tokenization schemes, pre-trained word embeddings,

and increasing Urdu finetuning data for the LLMs. Finally, to promote research, we will make UrFQuAD-v1 corpus publicly available.

ACKNOWLEDGMENT

This work was done with support from Dr. Ashraf Habibullah, president and CEO of Computers and Structures, Inc. (USA).

REFERENCES

- [1] D. Moldovan, S. Harabagiu, M. Pasca, R. Mihalcea, R. Girju, R. Goodrum, and V. Rus, "The structure and performance of an open-domain question answering system," in *Proc. 38th Annu. Meeting Assoc. Comput. Linguistics*, 2000, pp. 563–570, doi: [10.3115/1075218.1075289](https://doi.org/10.3115/1075218.1075289).
- [2] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 453–466, Nov. 2019, doi: [10.1162/tac1_a_00276](https://doi.org/10.1162/tac1_a_00276).
- [3] Y. Yang, W.-T. Yih, and C. Meek, "WikiQA: A challenge dataset for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 2013–2018, doi: [10.18653/v1/D15-1237](https://doi.org/10.18653/v1/D15-1237).
- [4] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," 2016, *arXiv:1606.05250*.
- [5] P. Rajpurkar, R. Jia, and P. Liang, "Know what you don't know: Unanswerable questions for SQuAD," 2018, *arXiv:1806.03822*.
- [6] M. Dunn, L. Sagun, M. Higgins, V. U. Guney, V. Cirik, and K. Cho, "Searchqa: A new q&a dataset augmented with context from a search engine," 2017, *arXiv:1704.05179*.
- [7] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," 2017, *arXiv:1705.03551*.
- [8] J. Liu, Y. Lin, Z. Liu, and M. Sun, "XQA: A cross-lingual open-domain question answering dataset," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 2358–2368, doi: [10.18653/v1/P19-1227](https://doi.org/10.18653/v1/P19-1227).
- [9] K. Darvishi, N. Shahbodaghkhan, Z. Abbasiantaeb, and S. Momtazi, "PQuAD: A Persian question answering dataset," *Comput. Speech Lang.*, vol. 80, May 2023, Art. no. 101486, doi: [10.1016/j.csl.2023.101486](https://doi.org/10.1016/j.csl.2023.101486).
- [10] K. Van Nguyen, D.-V. Nguyen, A. G.-T. Nguyen, and N. L.-T. Nguyen, "A Vietnamese dataset for evaluating machine reading comprehension," 2020, *arXiv:2009.14725*.
- [11] R. Obeidat, M. Al-Harbi, M. Al-Ayyoub, and L. Alawneh, "ArQuAD: An expert-annotated Arabic machine reading comprehension dataset," *Cognit. Comput.*, vol. 16, no. 3, pp. 984–1003, May 2024, doi: [10.1007/s12559-024-10248-6](https://doi.org/10.1007/s12559-024-10248-6).
- [12] S. Kazi and S. Khoja, "UQuAD1.0: Development of an Urdu question answering training data for machine reading comprehension," 2021, *arXiv:2111.01543*.
- [13] S. Arif, S. Farid, A. Athar, and A. A. Raza, "UQA: Corpus for Urdu question answering," 2024, *arXiv:2405.01458*.
- [14] A. Daud, W. Khan, and D. Che, "Urdu language processing: A survey," *Artif. Intell. Rev.*, vol. 47, no. 3, pp. 279–311, Mar. 2017, doi: [10.1007/s10462-016-9482-x](https://doi.org/10.1007/s10462-016-9482-x).
- [15] L. Yu, K. Moritz Hermann, P. Blunsom, and S. Pulman, "Deep learning for answer sentence selection," 2014, *arXiv:1412.1632*.
- [16] C. Clark and M. Gardner, "Simple and effective multi-paragraph reading comprehension," 2017, *arXiv:1710.10723*.
- [17] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," 2018, *arXiv:1802.05365*.
- [18] R. Kadlec, M. Schmid, O. Bajgar, and J. Kleindienst, "Text understanding with the attention sum reader network," 2016, *arXiv:1603.01547*.
- [19] M. Seo, A. Kembhavi, A. Farhadi, and H. Hajishirzi, "Bidirectional attention flow for machine comprehension," 2016, *arXiv:1611.01603*.
- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [21] P. Lewis, B. Oğuz, R. Rinott, S. Riedel, and H. Schwenk, "MLQA: Evaluating cross-lingual extractive question answering," 2019, *arXiv:1910.07475*.

- [22] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," 2019, *arXiv:1911.02116*.
- [23] S. Longpre, Y. Lu, and J. Daiber, "MKQA: A linguistically diverse benchmark for multilingual open domain question answering," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 1389–1406, Dec. 2021, doi: [10.1162/tac1_a_00433](https://doi.org/10.1162/tac1_a_00433).
- [24] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based model for Arabic language understanding," 2020, *arXiv:2003.00104*.
- [25] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, "MT5: A massively multilingual pre-trained text-to-text transformer," 2020, *arXiv:2010.11934*.
- [26] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense passage retrieval for open-domain question answering," 2020, *arXiv:2004.04906*.
- [27] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [28] M. Gabburo, S. Campese, F. Agostini, and A. Moschitti, "Datasets for multilingual answer sentence selection," 2024, *arXiv:2406.10172*.
- [29] Z. Duan, X. Li, Z. Zhang, Z. Li, N. Liu, and J. Wang, "Bridging the language gap: Knowledge injected multilingual question answering," in *Proc. IEEE Int. Conf. Big Knowl. (ICBK)*, Dec. 2021, pp. 339–346, doi: [10.1109/ICKG52313.2021.00052](https://doi.org/10.1109/ICKG52313.2021.00052).
- [30] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "AmbigQA: Answering ambiguous open-domain questions," 2020, *arXiv:2004.10645*.
- [31] M. Lamm, J. Palomaki, C. Alberti, D. Andor, E. Choi, L. Baldini Soares, and M. Collins, "QED: A framework and dataset for explanations in question answering," 2020, *arXiv:2009.06354*.
- [32] E. Choi, H. He, M. Iyyer, M. Yatskar, W.-t. Yih, Y. Choi, P. Liang, and L. Zettlemoyer, "QuAC : Question answering in context," 2018, *arXiv:1808.07036*.
- [33] J. H. Clark, E. Choi, M. Collins, D. Garrette, T. Kwiatkowski, V. Nikolaev, and J. Palomaki, "TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages," *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 454–470, Jul. 2020.
- [34] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary, and T. Wang, "MS MARCO: A human generated Machine reading COmprehension dataset," 2016, *arXiv:1611.09268*.
- [35] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading Wikipedia to answer open-domain questions," 2017, *arXiv:1704.00051*.
- [36] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009, doi: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [37] K. S. Jones, "A statistical interpretation of term specificity and its application in retrieval," *J. Document.*, vol. 28, no. 1, pp. 11–21, Jan. 1972, doi: [10.1108/eb026526](https://doi.org/10.1108/eb026526).
- [38] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2021, pp. 2288–2292, doi: [10.1145/3404835.3463098](https://doi.org/10.1145/3404835.3463098).
- [39] S. Jain, M. van Zuylen, H. Hajishirzi, and I. Beltagy, "SciREX: A challenge dataset for document-level information extraction," 2020, *arXiv:2005.00512*.
- [40] P. Mandikal and R. Mooney, "Sparse meets dense: A hybrid approach to enhance scientific document retrieval," 2024, *arXiv:2401.04055*.
- [41] X. Ma, K. Sun, R. Pradeep, and J. Lin, "A replication study of dense passage retriever," 2021, *arXiv:2104.05740*.
- [42] Z. Wang, P. Ng, X. Ma, R. Nallapati, and B. Xiang, "Multi-passage BERT: A globally normalized BERT model for open-domain question answering," 2019, *arXiv:1908.08167*.
- [43] L. Pan, C.-W. Hang, A. Sil, and S. Potdar, "Improved text classification via contrastive adversarial training," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, pp. 11130–11138, doi: [10.1609/aaai.v36i10.21362](https://doi.org/10.1609/aaai.v36i10.21362).
- [44] Y. Gong, L. Mao, and C. Li, "Few-shot learning for named entity recognition based on BERT and two-level model fusion," *Data Intell.*, vol. 3, no. 4, pp. 568–577, Oct. 2021, doi: [10.1162/dint_a_00102](https://doi.org/10.1162/dint_a_00102).
- [45] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [46] S. Ma, J. Yang, H. Huang, Z. Chi, L. Dong, D. Zhang, H. H. Awadalla, A. Muzio, A. Eriguchi, S. Singhal, X. Song, A. Menezes, and F. Wei, "XLM-T: Scaling up multilingual machine translation with pretrained cross-lingual transformer encoders," 2020, *arXiv:2012.15547*.
- [47] A. Kumar and V. H. C. Albuquerque, "Sentiment analysis using XLM-R transformer and zero-shot transfer learning on resource-poor Indian language," *ACM Trans. Asian Low-Resource Lang. Inf. Process.*, vol. 20, no. 5, pp. 1–13, Sep. 2021, doi: [10.1145/3461764](https://doi.org/10.1145/3461764).
- [48] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," 2020, *arXiv:2004.09813*.
- [49] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," 2019, *arXiv:1908.10084*.
- [50] P. Lewis, E. Perez, A. Piktou, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [51] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, Feb. 2019.
- [52] OpenAI et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [53] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
- [54] T. Chakraborty, V. La Gatta, V. Moscato, and G. Sperli, "Information retrieval algorithms and neural ranking models to detect previously fact-checked information," *Neurocomputing*, vol. 557, Nov. 2023, Art. no. 126680, doi: [10.1016/j.neucom.2023.126680](https://doi.org/10.1016/j.neucom.2023.126680).



MUHAMMAD SHAKEEL received the M.S. degree in software engineering from Australian National University, Australia. He is currently pursuing the Ph.D. degree in computer science with COMSATS University Islamabad, Lahore Campus. He is currently an experienced academic with over 20 years of teaching. He is also leading teams on AI software development at a U.S.-based software company. His research interests include natural language processing, deep learning, large language models, and information retrieval.



RAO MUHAMMAD ADEEL NAWAB received the Ph.D. degree in computer science from The University of Sheffield, U.K. He is currently an Assistant Professor with the Computer Science Department, COMSATS University Islamabad, Lahore Campus, Pakistan. His research interests include natural language processing, text reuse, plagiarism detection, author profiling, word sense disambiguation, text summarization, image captioning, information retrieval, question-answering systems, and paraphrase detection.

...